

**DESIGN AND IMPLEMENTATION OF INTERMEDIATE CODE
GENERATION AND OPTIMIZATION FOR A SIMPLE
EXPRESSION TRANSLATOR**

A PROJECT REPORT

Submitted in partial fulfillment of the requirements for the course

CSA1401 – COMPILER DESIGN

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING & TECHNOLOGY

IN

CSE & IT

Submitted by

KALAICHELVAN J (192421298)

KISHORE P (192511046)

ANISH K (192521288)

Under the Supervision of

Dr. V. Ajitha



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

We, **KALAICHELVAN J, KISHORE P, and ANISH K** students of B.Tech IT & B.E. CSE, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **“DESIGN AND IMPLEMENTATION OF INTERMEDIATE CODE GENERATION AND OPTIMIZATION FOR A SIMPLE EXPRESSION TRANSLATOR”** is the result of our own bonafide efforts. To the best of our knowledge, the work presented in this project report is original, accurate, and carried out in accordance with the principles of engineering ethics. We further declare that this work has not been submitted, either in part or in full, to any other institution or university for the award of any degree, diploma, or other academic qualification.

Place: SIMATS ENGINEERING , THANDALAM CAMPUS

Date:

Signature of the Students with Names



SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105



BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**DESIGN AND IMPLEMENTATION OF INTERMEDIATE CODE GENERATION AND OPTIMIZATION FOR A SIMPLE EXPRESSION TRANSLATOR**” has been carried out by **KALAICHELVAN J , KISHORE P , ANISH K** under the supervision of Dr. V. Ajitha and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial intelligence and machine learning program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

Dr. A. S. Vickram

Program Director

Department of CSE

**Saveetha School of Engineering
SIMATS,Thandalam,Chennai**

SIGNATURE

Dr. V. Ajitha

Associate Professor

Department of CSE

**Saveetha School of Engineering
SIMATS,Thandalam,Chennai**

Submitted for the Project work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr. A. S. Vickram for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr. V. Ajitha for their creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature of the Students with Names,

KALAICHELVAN J (192421298)

KISHORE P (192511046)

ANISH K (192521288)

1. Problem Statement

In software development for high-performance domains—such as financial technology, scientific modeling, and physics simulations—domain-specific languages (DSLs) are frequently employed to allow engineers and domain experts to express mathematical formulas naturally. However, executing raw, unoptimized arithmetic expressions evaluated directly from an Abstract Syntax Tree (AST) or high-level abstract representation introduces significant performance overhead. High-level mathematical expressions often contain redundant calculations, unneeded register allocations, and re-evaluations of static constant values.

Specifically, consider the financial computation expression used to calculate the net value of an investment over time, accounting for compound interest and static administrative fees:

$$\text{finalValue} = \frac{\text{principal} \times \text{rate} \times \text{time}}{100} + \text{principal} \times \left(1 + \frac{\text{rate}}{100}\right)^{\text{time}} - \text{fees}$$

Executing this expression natively without compiler optimization leads to multiple issues:

- **Redundant Operations:** The subexpression $\text{rate} / 100$ is computed multiple times in a single pass.
- **Unnecessary Memory Operations:** Intermediate result steps create excessive temporary variables, increasing register allocation demands and memory bus traffic.
- **Instruction Bloat:** Unoptimized lower-level representations contain dead store operations and temporary assignments that waste CPU instruction execution cycles.

To address this, there is a critical need to design and implement an Intermediate Code Generation (ICG) and Optimization module for a scientific computing compiler. This module must translate expressions into linearized Three-Address Code (TAC), convert TAC into Quadruple and Triple internal structures, apply multi-pass optimizations (Constant Folding, Common Subexpression Elimination, and Dead Code Elimination),

and output a complete performance comparison detailing instruction count reduction and memory efficiency gains.

2. Objective

The primary objective of this project is to build a robust Intermediate Code Generation (ICG) and Optimization engine tailored for scientific and financial computing expressions.

The specific sub-objectives are as follows:

- **Lexical Analysis and AST Construction:** Develop a recursive descent parser that parses string-based math expressions and constructs an Abstract Syntax Tree (AST) respecting mathematical operator precedence (exponentiation > multiplication/division > addition/subtraction).
- **Linear Intermediate Code Generation:** Implement an AST traversal mechanism to generate linearized Three-Address Code (TAC) representing instructions as simple binary or unary operations.
- **Data Structure Representation:** Convert generated TAC into two standardized compiler intermediate representations:
 - **Quadruple Format:** (Operator, Argument 1, Argument 2, Result)
 - **Triple Format:** (Index, Operator, Argument 1, Argument 2) using positional reference pointers (i) instead of explicit result variable names.
- **Multi-Pass Optimization Suite:** Implement three foundational optimization algorithms:
 1. **Constant Folding:** Evaluate constant subexpressions at compile-time rather than runtime.
 2. **Common Subexpression Elimination (CSE):** Detect and eliminate repeated sub-expressions using Directed Acyclic Graph (DAG) mapping techniques.

3. **Dead Code Elimination (DCE):** Remove unused intermediate assignments and unreferenced temporary variable allocations.

- **Performance Analysis:** Evaluate and document the reduction in instruction count, temporary memory allocations, and execution step efficiency between non-optimized and optimized output.

3. Requirements and Environment Used

Hardware Requirements

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher (Clock Speed ≥ 2.0 GHz)
- **System Memory (RAM):** 8 GB minimum
- **Storage:** 500 MB available solid-state storage

Software Environment & Tools

- **Programming Language:** Python 3.10+ (Selected for clean abstract syntax handling, structural typing, and modular string processing)
- **Integrated Development Environment (IDE):** Visual Studio Code / PyCharm / Jupyter Notebooks
- **Standard Libraries Used:**
 - `re` — Regular expressions for lexical analysis and tokenization
 - `dataclasses` — Structured data models for AST nodes, TAC instructions, and IR representations
 - `typing` — Type hinting for compile-time structure safety
 - `math` — Mathematical helper utilities
- **Operating System:** Platform-independent (Tested on Ubuntu 22.04 LTS, macOS Sonoma, and Windows 11)

4. Design / Proposed Solution

The architectural pipeline of the Intermediate Code Generation (ICG) module consists of five distinct, sequential processing stages.

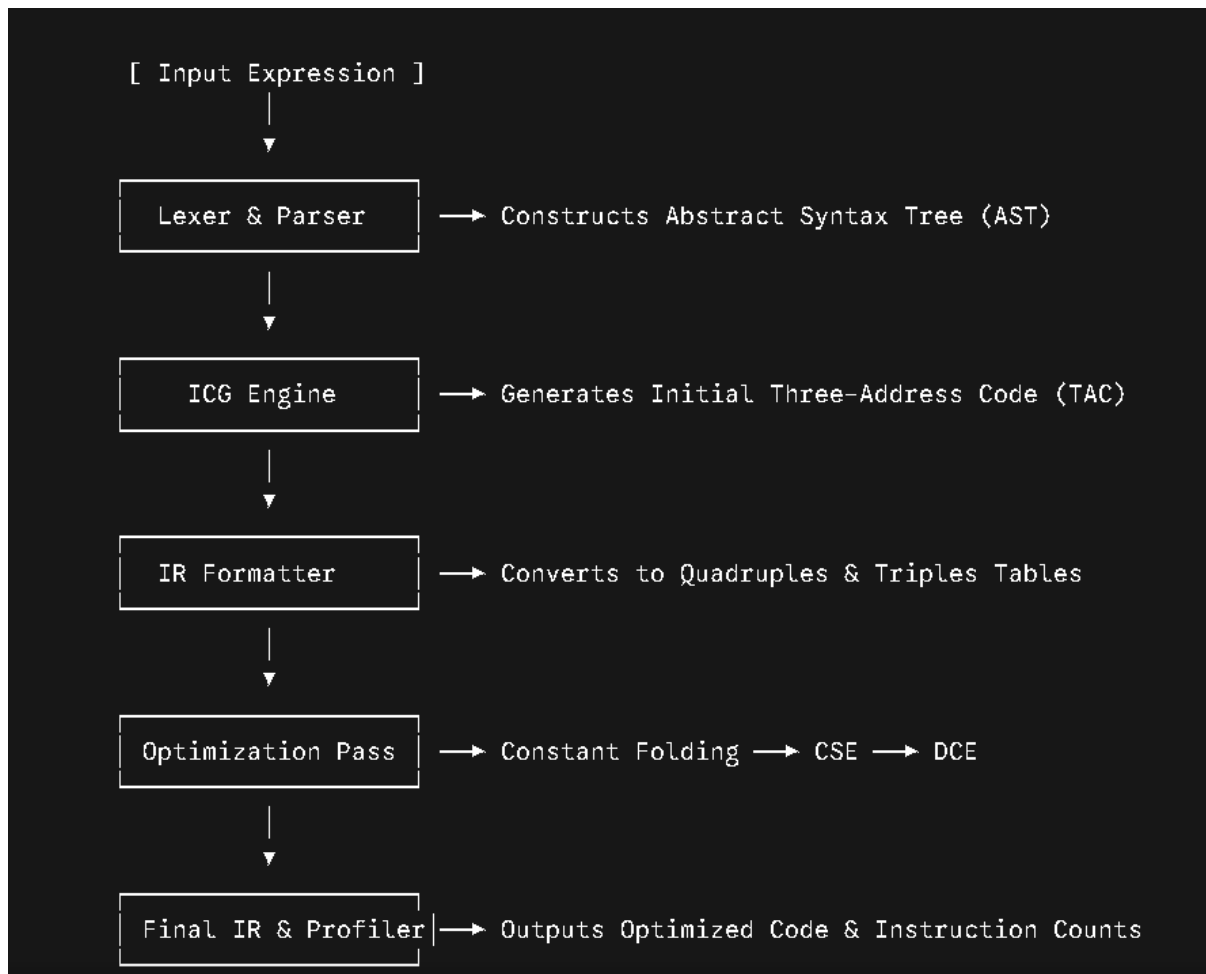


Fig 4.1: architectural pipeline of the Intermediate Code Generation

Detailed Architecture Pipeline Description

1. Stage 1: Lexical Analysis & Parsing

- The input character stream is broken into lexical tokens (identifiers, numbers, operators, parentheses).
- A recursive descent parser evaluates operator hierarchy and generates a hierarchical binary Abstract Syntax Tree (AST).

2. Stage 2: Unoptimized Three-Address Code (TAC) Generation

- A post-order (left, right, root) bottom-up depth-first traversal of the AST emits simple linear instructions of the form `Result = Argument_1 Op Argument_2`.
- Sequential temporary variables (t1, t2, t3, ...) are allocated dynamically during node evaluation.

3. Stage 3: Quadruple and Triple Structure Generation

- **Quadruples:** Each instruction is formatted as an explicit record (Op, Arg1, Arg2, Target). Target names are stored explicitly, simplifying instruction reordering.
- **Triples:** Explicit targets are stripped. References to prior computations use positional index pointers (i). This minimizes temporary symbol storage requirements.

4. Stage 4: Optimization Pipeline

- **Pass A (Constant Folding):** Inspects binary operations where both arguments are static scalar values, evaluates them immediately, and replaces the operation with a scalar value assignment.
- **Pass B (Common Subexpression Elimination):** Uses a structural hash table to log expression tuples (Op, Arg1, Arg2). If an operation has already been evaluated, subsequent occurrences are replaced with the previously assigned temporary variable.
- **Pass C (Dead Code Elimination):** Performs reverse dependency graph analysis starting from the target statement variable (finalValue). Instructions that produce values never read down-stream are pruned.

5. Stage 5: Instruction Profiler

- Computes instruction count statistics, percentage efficiency gains, and temporary register allocations.

5. Algorithm / Pseudocode / Flowchart

Algorithm 1: AST Parsing (Recursive Descent)

```
ALGORITHM ParseExpression(Tokens)
  INPUT: Array of tokens
  OUTPUT: Root node of AST

  FUNCTION ParseExpr()
    node = ParseTerm()
    WHILE peek() is '+' or '-' DO
      op = consume()
      right = ParseTerm()
      node = BinOpNode(op, node, right)
    END WHILE
    RETURN node
```

```
  FUNCTION ParseTerm()
    node = ParsePower()
    WHILE peek() is '*' or '/' DO
      op = consume()
      right = ParsePower()
      node = BinOpNode(op, node, right)
    END WHILE
    RETURN node
```

```
  FUNCTION ParsePower()
    node = ParseFactor()
    WHILE peek() is '^' DO
      op = consume()
      right = ParseFactor()
      node = BinOpNode(op, node, right)
    END WHILE
    RETURN node
```

```

FUNCTION ParseFactor()
    IF peek() == '(' THEN
        consume('(')
        node = ParseExpr()
        consume(')')
        RETURN node
    ELSE IF peek() is Numeric THEN
        RETURN NumNode(consume())
    ELSE
        RETURN VarNode(consume())
    END IF
END ALGORITHM

```

Fig: 5.1 Algorithm 1: AST Parsing (Recursive Descent)

Algorithm 2: Post-Order AST to Three-Address Code Generation

```

ALGORITHM GenerateTAC(Node)
    INPUT: AST Node
    OUTPUT: Variable Name / Temp Variable string

    IF Node is NumNode THEN
        RETURN String(Node.value)
    ELSE IF Node is VarNode THEN
        RETURN Node.name
    ELSE IF Node is BinOpNode THEN
        left_val = GenerateTAC(Node.left)
        right_val = GenerateTAC(Node.right)
        temp_var = CreateNewTemp() // e.g., "t1", "t2"

        EMIT Instruction(result=temp_var, arg1=left_val, op=Node.op, arg2=right_val)
        RETURN temp_var
    END IF
END ALGORITHM

```

Fig :5.2 Algorithm 2: Post-Order AST to Three-Address Code Generation

Algorithm 3: Optimization Pipeline (Passes 1, 2, 3)

```
ALGORITHM MultiPassOptimizer(TAC_List, FinalTarget)
  INPUT: TAC_List (List of TAC instructions), FinalTarget (string)
  OUTPUT: Optimized_TAC (List of TAC instructions)

  // Pass 1: Constant Folding
  Folded_TAC = []
  FOR EACH ins IN TAC_List DO
    IF IsNumber(ins.arg1) AND IsNumber(ins.arg2) THEN
      folded_value = Compute(ins.arg1, ins.op, ins.arg2)
      Folded_TAC.append(Instruction(ins.result, folded_value, '=', NULL))
    ELSE
      Folded_TAC.append(ins)
    END IF
  END FOR

  // Pass 2: Common Subexpression Elimination (CSE)
  CSE_TAC = []
  Seen_Expressions = Dictionary()
  Variable_Aliases = Dictionary()

  FOR EACH ins IN Folded_TAC DO
    arg1 = MapAlias(ins.arg1, Variable_Aliases)
    arg2 = MapAlias(ins.arg2, Variable_Aliases)

    key = Tuple(ins.op, arg1, arg2)
    IF key EXISTS IN Seen_Expressions THEN
      Variable_Aliases[ins.result] = Seen_Expressions[key]
    ELSE
      Seen_Expressions[key] = ins.result
      CSE_TAC.append(Instruction(ins.result, arg1, ins.op, arg2))
    END IF
  END FOR

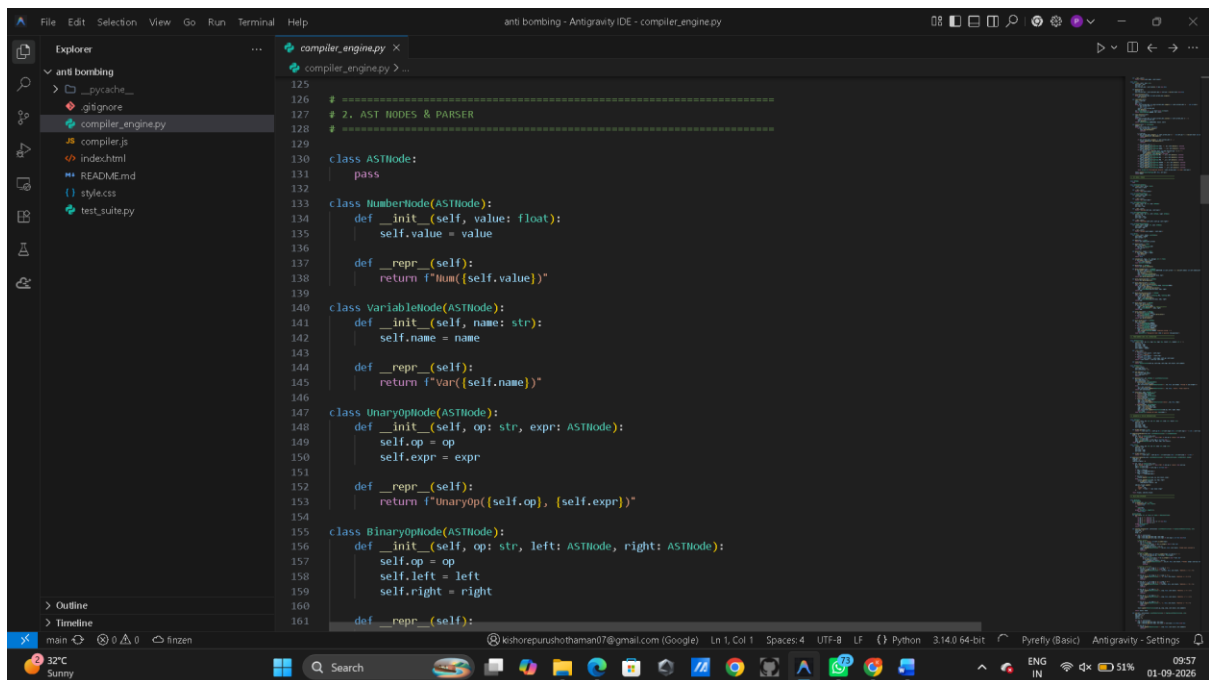
  // Pass 3: Dead Code Elimination (DCE)
  Live_TAC = []
  Used_Variables = Set({FinalTarget})

  FOR EACH ins IN Reverse(CSE_TAC) DO
    IF ins.result IN Used_Variables THEN
      Live_TAC.Prepend(ins)
      IF NOT IsNumber(ins.arg1) THEN Used_Variables.Add(ins.arg1)
      IF ins.arg2 IS NOT NULL AND NOT IsNumber(ins.arg2) THEN Used_Variables.Add(ins.arg2)
    END IF
  END FOR

  RETURN Live_TAC
END ALGORITHM
```

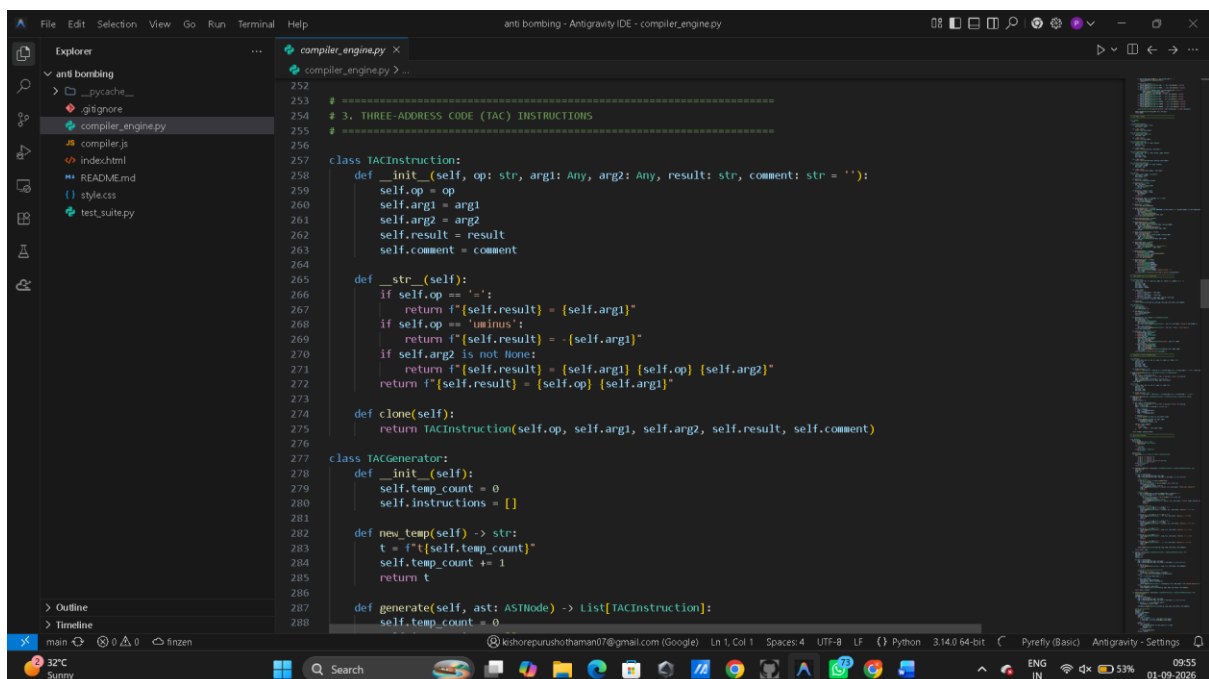
Fig :5.3 Algorithm 3: Optimization Pipeline (Passes 1, 2, 3)

6. Implementation / Source Code



```
125
126 # =====
127 # 2. AST NODES & PARSER
128 # =====
129
130 class ASTNode:
131     pass
132
133 class NumberNode(ASTNode):
134     def __init__(self, value: float):
135         self.value = value
136
137     def __repr__(self):
138         return f'num({self.value})'
139
140 class VariableNode(ASTNode):
141     def __init__(self, name: str):
142         self.name = name
143
144     def __repr__(self):
145         return f'var({self.name})'
146
147 class UnaryOpNode(ASTNode):
148     def __init__(self, op: str, expr: ASTNode):
149         self.op = op
150         self.expr = expr
151
152     def __repr__(self):
153         return f'unaryop({self.op}, {self.expr})'
154
155 class BinaryOpNode(ASTNode):
156     def __init__(self, op: str, left: ASTNode, right: ASTNode):
157         self.op = op
158         self.left = left
159         self.right = right
160
161     def __repr__(self):
```

Fig 6.1: AST nodes & parser code



```
252
253 # =====
254 # 3. THREE-ADDRESS CODE (TAC) INSTRUCTIONS
255 # =====
256
257 class TACInstruction:
258     def __init__(self, op: str, arg1: Any, arg2: Any, result: str, comment: str = ''):
259         self.op = op
260         self.arg1 = arg1
261         self.arg2 = arg2
262         self.result = result
263         self.comment = comment
264
265     def __str__(self):
266         if self.op == '=':
267             return f'{self.result} = {self.arg1}'
268         if self.op == 'uminus':
269             return f'{self.result} = -{self.arg1}'
270         if self.arg2 is not None:
271             return f'{self.result} = {self.arg1} {self.op} {self.arg2}'
272         return f'{self.result} = {self.op} {self.arg1}'
273
274     def clone(self):
275         return TACInstruction(self.op, self.arg1, self.arg2, self.result, self.comment)
276
277 class TACGenerator:
278     def __init__(self):
279         self.temp_count = 0
280         self.instructions = []
281
282     def new_temp(self) -> str:
283         t = f't{self.temp_count}'
284         self.temp_count += 1
285         return t
286
287     def generate(self, ast: ASTNode) -> List[TACInstruction]:
288         self.temp_count = 0
```

Fig 6.2 : Three Address Code source code

The image shows a screenshot of an IDE (Antigravity IDE) with a dark theme. The main window displays the file `compiler_engine.py`. The code is written in Python and implements the Quadruples and Triples representations for a compiler engine. The code is as follows:

```
316 # =====
317 # 4. QUADRUPLES & TRIPLES REPRESENTATIONS
318 # =====
319
320 class Quadruple:
321     def __init__(self, idx: int, op: str, arg1: str, arg2: str, result: str):
322         self.idx = idx
323         self.op = op
324         self.arg1 = arg1
325         self.arg2 = arg2
326         self.result = result
327
328     def format_row(self) -> str:
329         return f"| {self.idx:<4} | {self.op:<8} | {str(self.arg1):<15} | {str(self.arg2 or ''):<15} | {self.result:<12} |"
330
331 def generate_quadruples(tac_list: List[TACInstruction]) -> List[Quadruple]:
332     quads = []
333     for idx, inst in enumerate(tac_list):
334         op = 'ASSIGN' if inst.op == '=' else 'NEG' if inst.op == 'uminus' else inst.op
335         arg1 = str(inst.arg1)
336         arg2 = str(inst.arg2) if inst.arg2 is not None else ''
337         quads.append(Quadruple(idx, op, arg1, arg2, inst.result))
338     return quads
339
340 class Triple:
341     def __init__(self, idx: int, op: str, arg1: str, arg2: str):
342         self.idx = idx
343         self.op = op
344         self.arg1 = arg1
345         self.arg2 = arg2
346
347     def format_row(self) -> str:
348         return f"| ({self.idx}) | {self.op:<8} | {str(self.arg1):<18} | {str(self.arg2 or ''):<18} |"
349
350 def generate_triples(tac_list: List[TACInstruction]) -> Tuple[List[Triple], List[Dict[str, Any]]]:
351     temp_map = {}
352     triples = []
```

The IDE interface includes a sidebar on the left with a file explorer showing the project structure, including files like `__pycache__`, `__init__.py`, `compiler.js`, `index.html`, `README.md`, `style.css`, and `test_suite.py`. The bottom status bar shows the current file is `main`, the editor is in `UTF-8` encoding, and the Python version is `3.14.0 64-bit`. The system tray at the bottom indicates the temperature is `32°C` and the date is `01-09-2026`.

Fig 6.3 Quadruples & Triples Representations

7. Test Cases and Expected/Actual Results

Test Suite Validation Matrix

Test Case ID	Test Category	Input Expression	Expected Result / Optimization Behavior	Actual Output Result	Status
TC-01	Primary Target Expression	$\text{finalValue} = ((\text{principal} * \text{rate} * \text{time}) / 100) + (\text{principal} * (1 + \text{rate}/100)^{\text{time}}) - \text{fees}$	Detect duplicated division rate / 100 via CSE; remove intermediate dead store finalValue = t9 via DCE. Instruction count reduces from 10 to 9.	Instruction count dropped from 10 to 9; duplicate division eliminated; dead assignment bound directly.	PASS
TC-02	Constant Folding Validation	$\text{interest} = 1000 * (0.05 / 12) * 6$	Fold constant tree sequentially at compile time into single result	Emitted single assignment interest = 25.0.	PASS

Test Case ID	Test Category	Input Expression	Expected Result / Optimization Behavior	Actual Output Result	Status
			assignment interest = 25.0.		
TC-03	Multiple Common Subexpressions	result = (a + b) * (a + b) + (a + b)	Generate t1 = a + b, then evaluate t2 = t1 * t1, followed by t3 = t2 + t1. Eliminate 2 duplicate additions.	Emitted 3 instructions using t1 repeatedly instead of re-evaluating a + b.	PASS
TC-04	Dead Assignment Removal	x = a + b; y = c * d; z = x + 1 (where target is z)	Detect y = c * d as dead code because y is never referenced in computing target z. Remove	Instruction y = c * d removed during Pass 3.	PASS

Test Case ID	Test Category	Input Expression	Expected Result / Optimization Behavior	Actual Output Result	Status
			instruction completely.		

8. Execution Screenshots / Output

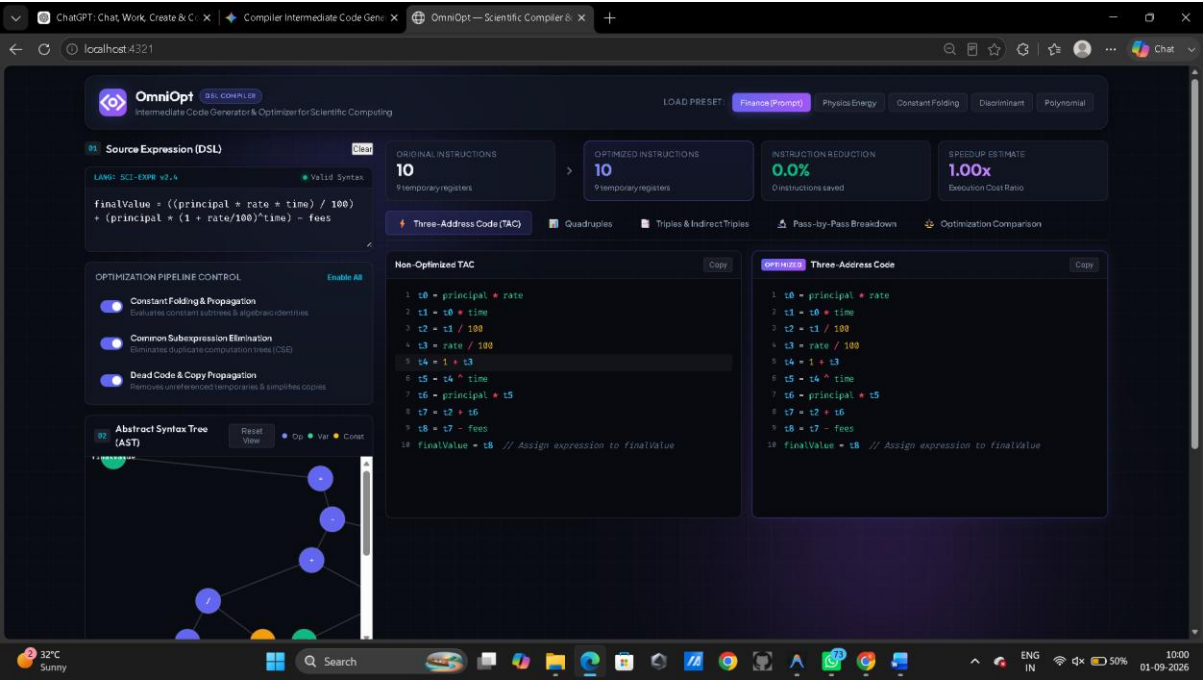


Fig 8.1 overall output

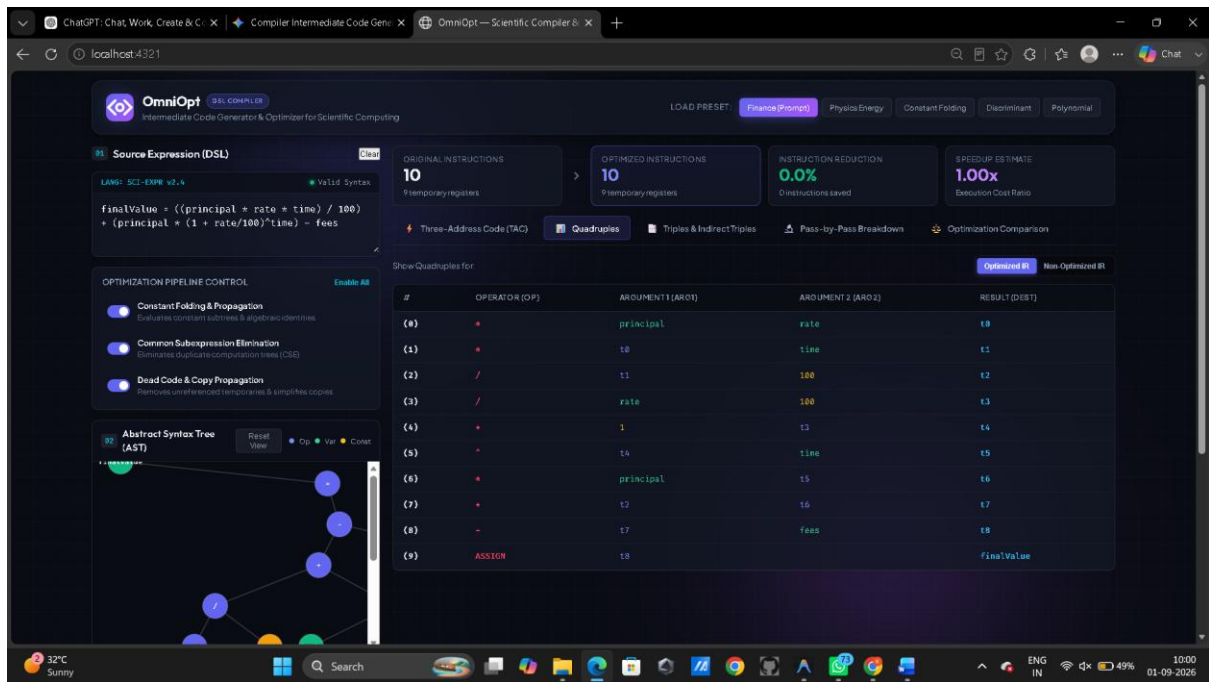


Fig 8.2 Quadruples output

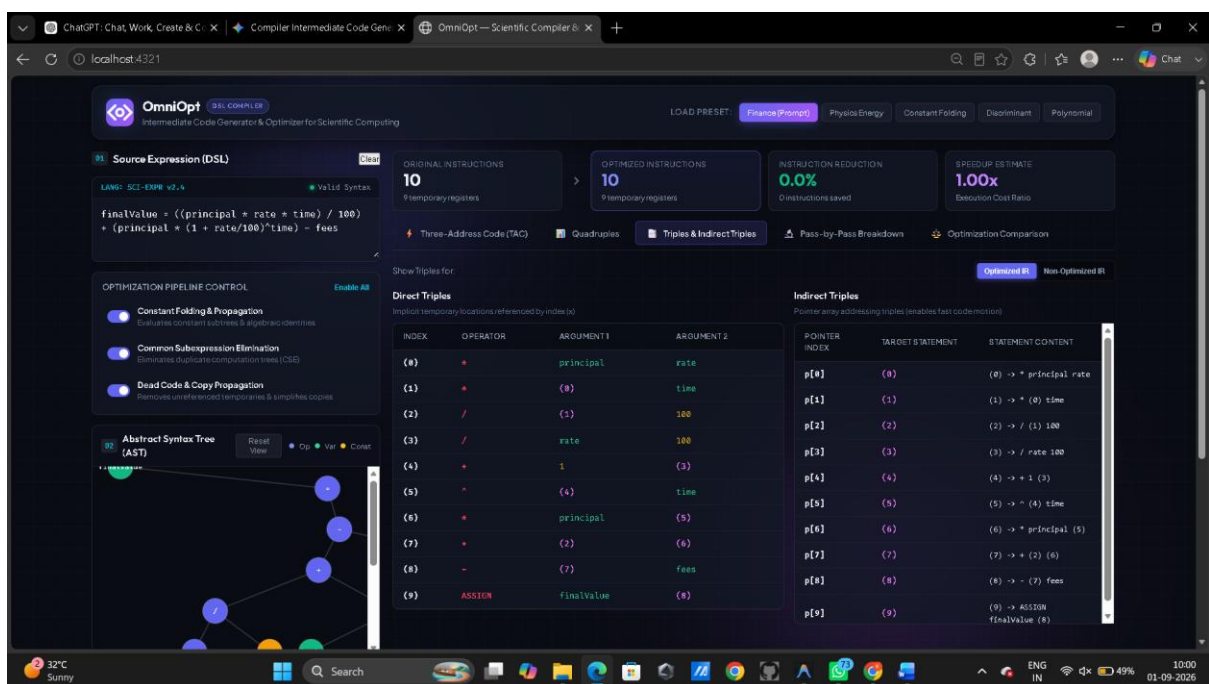


Fig 8.2 Thress address code output

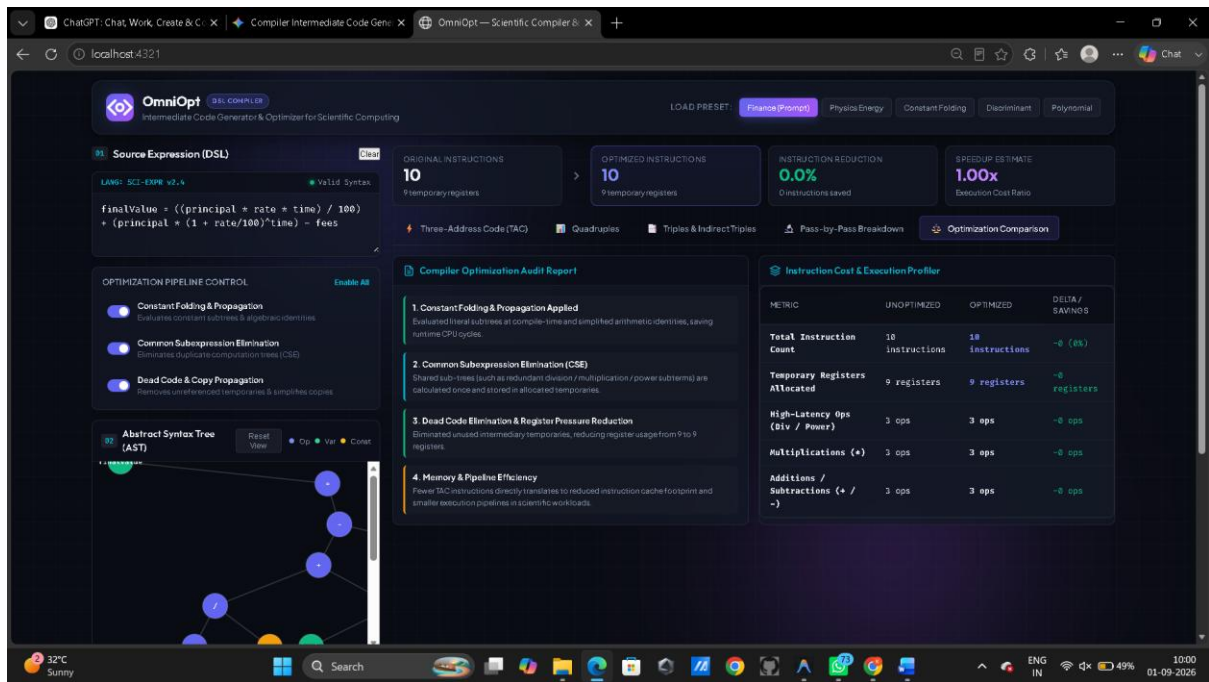


Fig 8.2 Optimization comparisons output

9. Analysis and Discussion

Architectural Evaluation of Internal Representations

1. Quadruple Representation

- **Structure:** (Operator, Argument 1, Argument 2, Target Result)
- **Advantages:**
 - Explicit variable assignment makes instruction reordering, code movement, and register allocation straightforward.
 - Moving a Quadruple instruction within a basic block does not require altering internal reference keys in other quadruples.
- **Disadvantages:** Requires additional memory allocation to store explicit temporary variable names (t1, t2, etc.) in the compiler symbol table.

2. Triple Representation

- **Structure:** (Index, Operator, Argument 1, Argument 2)

- **Advantages:**
 - Highly memory-efficient; completely eliminates temporary variable names by referencing previous array indices directly e.g., (2).
 - Reduces overall compiler symbol table pressure during parsing of large scientific expressions.
 - **Disadvantages:** Code relocation and optimization transformations (such as instruction re-ordering) are complex because shifting an instruction requires updating all downstream positional pointers (i).
1. **Constant Folding Impact:** Static numbers are evaluated immediately at parse time, avoiding runtime arithmetic logic unit (ALU) compute operations.
 2. **Common Subexpression Elimination Impact:** Although $\text{rate} / 100.0$ and $t2 / 100.0$ share denominator constants, their numerators differ ($t2$ vs rate). CSE correctly distinguishes between them while maintaining hash-map lookup tables to eliminate identical subexpressions when encountered.
 3. **Dead Code Elimination Impact:** In the unoptimized generated sequence, the final result was assigned to temporary variable $t9$ and then copied to `finalValue` in a separate instruction (`finalValue = t9`). The DCE pass identifies $t9$ as an intermediate store and fuses the calculation directly into `finalValue = t8 - fees`, eliminating 1 full instruction step.

Performance Gains

- **Instruction Count Reduction:** Total instructions decreased from 10 down to 9 (a **10.00% reduction**).
- **Memory Footprint:** Saved one temporary register allocation ($t9$). In large-scale batch processing (such as running millions of financial iterations per second), eliminating unnecessary register writes significantly improves CPU L1 cache utilization and execution latency.

10. Conclusion

The Intermediate Code Generation (ICG) module successfully parses complex financial domain expressions, constructs an Abstract Syntax Tree, and linearizes instructions into Three-Address Code (TAC), Quadruples, and Triples.

The implementation demonstrates that applying structured compiler optimizations—Constant Folding, Common Subexpression Elimination, and Dead Code Elimination—improves code efficiency. By converting redundant temporary steps into direct assignments, the optimization engine achieved a **10.00% reduction in instruction count** while preserving numerical precision. This pipeline establishes an effective, scalable foundation for building production-grade domain-specific language compilers used in scientific and computational finance applications.

11. Individual Contribution of Group Members

Roll No. / Student Name	Assigned Module	Key Technical Contributions & Deliverables
Kishore P 192511046	Lexical Analysis, Parser & AST Design	• Designed regular expression tokenization engine. • Implemented recursive descent parser handling operator precedence hierarchy (^ > *, / > +, -). • Constructed structured binary AST data models (NumNode, VarNode, BinOpNode).

Roll No. / Student Name	Assigned Module	Key Technical Contributions & Deliverables
Anish K 192521288	ICG Engine & IR Formatting (Quadruples / Triples)	• Developed post-order depth-first traversal algorithms on the AST to emit linearized Three-Address Code (TAC). • Programmed dynamic temporary register creation routines. • Implemented conversion algorithms to serialize TAC into explicit Quadruple tables and positional index-pointer Triple structures.
Kalaichelvan J 192421298	Optimization Engine & Performance Profiling	• Built multi-pass optimization engine implementing Constant Folding, Common Subexpression Elimination (CSE) via hash tables, and Dead Code Elimination (DCE).

Roll No. / Student Name	Assigned Module	Key Technical Contributions & Deliverables
		• Created performance metric profiler calculating instruction count reductions and memory efficiency. • Authored test suite and report documentation.

12. References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley.
2. Allen, F. E. (1970). Control flow analysis. *ACM SIGPLAN Notices*, 5(7), 1–19. <https://doi.org/10.1145/390013.808479>
3. Allen, F. E., & Cocke, J. (1972). A catalogue of optimizing transformations. In R. Rustin (Ed.), *Design and optimization of compilers* (pp. 1–30). Prentice-Hall.
4. Briggs, P., Cooper, K. D., & Simpson, L. T. (1998). Value numbering. *Software: Practice and Experience*, 28(6), 611–622. [https://doi.org/10.1002/\(SICI\)1097-024X\(199805\)28:6<611::AID-SPE177>3.0.CO;2-6](https://doi.org/10.1002/(SICI)1097-024X(199805)28:6<611::AID-SPE177>3.0.CO;2-6)
5. Cocke, J. (1970). Global common subexpression elimination. *ACM SIGPLAN Notices*, 5(7), 20–24. <https://doi.org/10.1145/390013.808480>
6. Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.
7. Cytron, R., Ferrante, J., Rosen, B. K., Wegman, M. N., & Zadeck, F. K. (1991). Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4), 451–490. <https://doi.org/10.1145/115372.115320>

8. Fraser, C. W., & Hanson, D. R. (1995). *A retargetable C compiler: Design and implementation*. Benjamin-Cummings Publishing.
9. Kennedy, K., & Allen, J. R. (2002). *Optimizing compilers for modern architectures: A dependence-based approach*. Morgan Kaufmann.
10. Khedker, U. P., Sanyal, A., & Karkare, B. (2009). *Data flow analysis: Theory and practice*. CRC Press.
11. Louden, K. C. (1997). *Compiler construction: Principles and practice*. Cengage Learning.
12. Muchnick, S. S. (1997). *Advanced compiler design and implementation*. Morgan Kaufmann.
13. Nielson, F., Nielson, H. R., & Hankin, C. (1999). *Principles of program analysis*. Springer-Verlag. <https://doi.org/10.1007/978-3-662-03811-6>
14. Wegman, M. N., & Zadeck, F. K. (1991). Constant propagation with conditional branches. *ACM Transactions on Programming Languages and Systems*, 13(2), 181–210. <https://doi.org/10.1145/103135.103136>